

Lab 07

Task-1:

We want to design a small database to track sailors, boats, and which boats are reserved by which sailor on a given day.

Database Creation:

```
1 CREATE TABLE Sailors (  
2   sid INT PRIMARY KEY,  
3   sname VARCHAR(50),  
4   rating INT,  
5   age FLOAT  
6 );  
7 CREATE TABLE Boats (  
8   bid INT PRIMARY KEY,  
9   bname VARCHAR(50),  
10  color VARCHAR(20)  
11 );  
12 CREATE TABLE Reserves (  
13   sid INT,  
14   bid INT,  
15   day DATE,  
16   PRIMARY KEY (sid, bid, day),  
17   FOREIGN KEY (sid) REFERENCES Sailors(sid),  
18   FOREIGN KEY (bid) REFERENCES Boats(bid)  
19 );  
20 INSERT INTO Sailors (sid, sname, rating, age) VALUES  
21 (1, 'Bob', 10, 22.5),  
22 (2, 'Alice', 8, 19.0),  
23 (3, 'Charlie', 7, 25.0),  
24 (4, 'David', 9, 20.0),  
25 (5, 'Eve', 8, 24.0);  
26 INSERT INTO Boats (bid, bname, color) VALUES  
27 (101, 'Boat1', 'red'),  
28 (102, 'Boat2', 'green'),  
29 (103, 'Boat3', 'blue'),  
30 (104, 'Boat4', 'red'),  
31 (105, 'Boat5', 'yellow');  
32 INSERT INTO Reserves (sid, bid, day) VALUES  
33 (1, 101, '2023-07-01'),  
34 (2, 102, '2023-07-02'),  
35 (3, 103, '2023-07-03'),  
36 (1, 104, '2023-07-04'),  
37 (4, 105, '2023-07-05'),  
38 (5, 101, '2023-07-06'),  
39 (1, 102, '2023-07-06'),  
40 (3, 102, '2023-07-06');
```

Queries

- Find all sailors who have reserved boats (sailor name + boat name).
- Show all sailors and the boats they reserved. If no reservation, show NULL for boat.
- Show all boats and the sailors who reserved them. If no one reserved a boat, show NULL for sailor.
- Show all sailors and all boats, even if no reservations exist.
- List all possible combinations of sailors and boats (Cartesian product).

```
SELECT * FROM Boats;
SELECT * FROM Sailors;
SELECT * FROM Reserves;
--Qno1:
SELECT S.sname , B.bname
FROM Sailors S
INNER JOIN Reserves R
ON S.sid=R.sid
INNER JOIN Boats B;
--Qno2:
SELECT S.sname, B.bname
FROM Sailors S
LEFT JOIN Reserves R ON S.sid = R.sid
LEFT JOIN Boats B ON R.bid = B.bid;
--Qno3:
SELECT B.bname, S.sname
FROM Boats B
LEFT JOIN Reserves R ON B.bid = R.bid
LEFT JOIN Sailors S ON R.sid = S.sid;
--Qno4:
SELECT S.sname , B.bname
FROM Sailors S
FULL OUTER JOIN Reserves R
ON S.sid=R.sid
FULL OUTER JOIN Boats B;
--Qno5:
SELECT S.sname ,B.bname
FROM Sailors S
CROSS JOIN Boats B;
```

output:

sname	bname
Bob	Boat1
Bob	Boat2
Bob	Boat3
Bob	Boat4
Bob	Boat5
Alice	Boat1
Alice	Boat2
Alice	Boat3
Alice	Boat4
Alice	Boat5
Charlie	Boat1
Charlie	Boat2
Charlie	Boat3
Charlie	Boat4
Charlie	Boat5
David	Boat1
David	Boat2
David	Boat3
David	Boat4
David	Boat5
Eve	Boat1

bname	sname
Boat1	Bob
Boat1	Eve
Boat2	Bob
Boat2	Alice
Boat3	Charlie
Boat4	Bob
Boat5	David

sname	bname
Bob	Boat1
Bob	Boat2
Bob	Boat4
Alice	Boat2
Charlie	Boat2
Charlie	Boat3
David	Boat5
Eve	Boat1

sname	bname
Bob	Boat1
Alice	Boat2
Charlie	Boat3
Bob	Boat4
David	Boat5
Eve	Boat1
Bob	Boat2
Charlie	Boat2

Task 2:

You are creating a database for an online shopping store like Daraz or Amazon. There are three tables:

Database Creation

```
CREATE TABLE Customers (  
  cust_id INT PRIMARY KEY,  
  cust_name VARCHAR(50),  
  city VARCHAR(50)  
);  
CREATE TABLE Products (  
  prod_id INT PRIMARY KEY,  
  prod_name VARCHAR(50),  
  category VARCHAR(30)  
);  
CREATE TABLE Orders (  
  order_id INT PRIMARY KEY,  
  cust_id INT,  
  prod_id INT,  
  order_date DATE,  
  FOREIGN KEY (cust_id) REFERENCES Customers(cust_id),  
  FOREIGN KEY (prod_id) REFERENCES Products(prod_id)  
);  
INSERT INTO Customers (cust_id, cust_name, city) VALUES  
(1, 'Ali', 'Karachi'),  
(2, 'Sara', 'Lahore'),  
(3, 'Ahmed', 'Islamabad'),  
(4, 'Hina', 'Karachi');  
INSERT INTO Products (prod_id, prod_name, category) VALUES  
(101, 'Laptop', 'Electronics'),  
(102, 'Mobile', 'Electronics'),  
(103, 'Shirt', 'Clothing'),  
(104, 'Shoes', 'Footwear');  
INSERT INTO Orders (order_id, cust_id, prod_id, order_date) VALUES  
(1001, 1, 101, '2023-09-01'),  
(1002, 2, 103, '2023-09-02'),  
(1003, 3, 102, '2023-09-03'),  
(1004, 1, 104, '2023-09-04');
```

Queries:

- Find which customers bought which products.

```
--Qno1:  
SELECT C.cust_name, P.prod_name  
FROM Orders O  
JOIN Customers C ON O.cust_id = C.cust_id  
JOIN Products P ON O.prod_id = P.prod_id;
```

Output:

! cust_name	prod_name
Ali	Laptop
Sara	Shirt
Ahmed	Mobile
Ali	Shoes

Show all customers and the products they ordered. If a customer hasn't placed an order, show NULL.

```
--Qno2:
SELECT C.cust_name, P.prod_name
FROM Customers C
LEFT JOIN Orders O ON C.cust_id = O.cust_id
LEFT JOIN Products P ON O.prod_id = P.prod_id;
--Qno3:
```

Output:

cust_name	prod_name
Ali	Laptop
Ali	Shoes
Sara	Shirt
Ahmed	Mobile
Hina	NULL

Show all products and the customers who ordered them. If no one ordered a product, show NULL

```
44 --Qno3:
45 SELECT P.prod_name, C.cust_name
46 FROM Products P
47 LEFT JOIN Orders O ON P.prod_id = O.prod_id
48 LEFT JOIN Customers C ON O.cust_id = C.cust_id;
```

prod_name	cust_name
Laptop	Ali
Mobile	Ahmed
Shirt	Sara
Shoes	Ali

Show all customers and all products, even if some products haven't been ordered or some customers haven't placed any order.

```
49 --Qno4:
50 SELECT C.cust_name, P.prod_name
51 FROM Customers C
52 CROSS JOIN Products P;
```

cust_name	prod_name
Ali	Laptop
Ali	Mobile
Ali	Shirt
Ali	Shoes
Sara	Laptop
Sara	Mobile
Sara	Shirt
Sara	Shoes
Ahmed	Laptop

Find customers who live in the same city

```
53 --Qno5:
54 SELECT C1.cust_name AS Customer1, C2.cust_name AS Customer2, C1.city
55 FROM Customers C1
56 JOIN Customers C2
57 ON C1.city = C2.city AND C1.cust_id < C2.cust_id;
```

Customer1	Customer2	city
Ali	Hina	Karachi

Generate all possible combinations of customers and products (who could buy what).

```
57 ON C1.city = C2.city AND C1.cust_id < C2.cust_id;
58 --Qno6:
59 SELECT C.cust_name, P.prod_name
60 FROM Customers C
61 CROSS JOIN Products P;
```

cust_name	prod_name
Ali	Laptop
Ali	Mobile
Ali	Shirt
Ali	Shoes
Sara	Laptop
Sara	Mobile
Sara	Shirt
Sara	Shoes
Ahmed	Laptop

Task 3:

You are designing a database for a movie streaming platform like Netflix.

Database Creation

```
1 CREATE TABLE Users (  
2   user_id INT PRIMARY KEY,  
3   user_name VARCHAR(50),  
4   city VARCHAR(50)  
5 );  
6 CREATE TABLE Movies (  
7   movie_id INT PRIMARY KEY,  
8   movie_title VARCHAR(100),  
9   genre VARCHAR(30)  
10 );  
11 CREATE TABLE WatchHistory (  
12   user_id INT,  
13   movie_id INT,  
14   watch_date DATE,  
15   PRIMARY KEY (user_id, movie_id, watch_date),  
16   FOREIGN KEY (user_id) REFERENCES Users(user_id),  
17   FOREIGN KEY (movie_id) REFERENCES Movies(movie_id)  
18 );  
19 INSERT INTO Users (user_id, user_name, city) VALUES  
20 (1, 'Ali', 'Karachi'),  
21 (2, 'Sara', 'Lahore'),  
22 (3, 'Ahmed', 'Islamabad'),  
23 (4, 'Hina', 'Karachi');  
24 INSERT INTO Movies (movie_id, movie_title, genre) VALUES  
25 (101, 'Inception', 'Sci-Fi'),  
26 (102, 'Titanic', 'Romance'),  
27 (103, 'The Dark Knight', 'Action'),  
28 (104, 'Inside Out', 'Animation');  
29 INSERT INTO WatchHistory (user_id, movie_id, watch_date) VALUES  
30 (1, 101, '2023-08-01'),  
31 (2, 102, '2023-08-02'),  
32 (3, 103, '2023-08-03'),  
33 (1, 104, '2023-08-04'),  
34 (2, 101, '2023-08-05');
```

Query:

- Find users and the movies they have watched.

```
35 --Query 1:  
36 SELECT U.user_name, M.movie_title  
37 FROM WatchHistory WH  
38 JOIN Users U ON WH.user_id = U.user_id  
39 JOIN Movies M ON WH.movie_id = M.movie_id;  
40
```

user_name	movie_title
Ali	Inception
Sara	Titanic
Ahmed	The Dark Knight
Ali	Inside Out
Sara	Inception

Show all users and the movies they watched. If a user hasn't watched anything, show NULL.

```

40 --Query 2:
41 SELECT U.user_name, M.movie_title
42 FROM Users U
43 LEFT JOIN WatchHistory WH ON U.user_id = WH.user_id
44 LEFT JOIN Movies M ON WH.movie_id = M.movie_id;

```

user_name	movie_title
Ali	Inception
Ali	Inside Out
Sara	Inception
Sara	Titanic
Ahmed	The Dark Knight
Hina	NULL

Show all movies and the users who watched them. If a movie hasn't been watched, show NULL

```

45 --Query 3:
46 SELECT M.movie_title, U.user_name
47 FROM Movies M
48 LEFT JOIN WatchHistory WH ON M.movie_id = WH.movie_id
49 LEFT JOIN Users U ON WH.user_id = U.user_id;

```

movie_title	user_name
Inception	Ali
Inception	Sara
Titanic	Sara
The Dark Knight	Ahmed
Inside Out	Ali

Show all users and all movies, even if no one watched the movie or if a user never watched anything.

```

50 --Query 4:
51 SELECT U.user_name, M.movie_title
52 FROM Users U
53 CROSS JOIN Movies M;

```

user_name	movie_title
Ali	Inception
Ali	Titanic
Ali	The Dark Knight
Ali	Inside Out
Sara	Inception
Sara	Titanic
Sara	The Dark Knight
Sara	Inside Out
Ahmed	Inception
Ahmed	Titanic
Ahmed	The Dark Knight
Ahmed	Inside Out
Hina	Inception
Hina	Titanic
Hina	The Dark Knight
Hina	Inside Out

Find pairs of users living in the same city

```
54 --Query 5:
55 SELECT U1.user_name AS User1, U2.user_name AS User2, U1.city
56 FROM Users U1
57 JOIN Users U2 ON U1.city = U2.city AND U1.user_id < U2.user_id;
```

User1	User2	city
Ali	Hina	Karachi

Generate all possible combinations of users and movies (who could watch which movie).

```
58 --Query 6:
59 SELECT U.user_name, M.movie_title
60 FROM Users U
61 CROSS JOIN Movies M;
```

user_name	movie_title
Ali	Inception
Ali	Titanic
Ali	The Dark Knight
Ali	Inside Out
Sara	Inception
Sara	Titanic
Sara	The Dark Knight
Sara	Inside Out
Ahmed	Inception
Ahmed	Titanic
Ahmed	The Dark Knight
Ahmed	Inside Out
Hina	Inception
Hina	Titanic
Hina	The Dark Knight
Hina	Inside Out